

GUÍA DE PRUEBAS

API REST de Canciones

Bruno + Laravel Sanctum + MySQL en VPS



Proyecto: EAGRCancionesAPI

Alumno: Edsai Alejandro García Reyes

URL de producción: <http://82.25.93.110:8082/api>

Objetivo: demostrar autenticación con tokens, operaciones CRUD y verificación directa en la base de datos del VPS.

Contenido de la guía

1. Contexto y arquitectura del proyecto
2. Preparación de Bruno y variables del entorno
3. Pruebas de autenticación
4. Pruebas CRUD de canciones
5. Validaciones y códigos de respuesta
6. Verificación de resultados en MySQL
7. Secuencia recomendada para exponer ante la maestra
8. Solución rápida de errores comunes

Importante antes de empezar

En la terminal copia solamente los comandos. No copies el texto del prompt, por ejemplo `root@srv...#` o `PS C:\...>`. Tampoco compartas contraseñas ni tokens en capturas.

1. Contexto y arquitectura del proyecto

La actividad consiste en probar una API REST de canciones creada con Laravel. La API se ejecuta en un VPS Ubuntu y usa Nginx, PHP-FPM, MySQL y Laravel Sanctum. Bruno funciona como cliente para enviar solicitudes HTTP y revisar las respuestas JSON.

Componente	Configuración
Cliente de pruebas	Bruno
Dirección base	<code>http://82.25.93.110:8082/api</code>
Servidor	VPS Ubuntu + Nginx + PHP 8.3
Backend	Laravel + Eloquent ORM
Autenticación	Laravel Sanctum con Bearer Token
Base de datos	MySQL: <code>eagr_canciones_api</code>
Usuario MySQL	<code>eagr_canciones</code>

2. Preparación de Bruno y variables del entorno

En la colección API Canciones EAGR crea dos carpetas: 01 Autenticación y 02 Canciones. Luego selecciona el ambiente VPS en la esquina superior derecha.

Variable	Valor de ejemplo	Uso
<code>baseUrl</code>	<code>http://82.25.93.110:8082/api</code>	Dirección base de la API.
<code>token</code>	<code>1 TOKEN_COMPLETO</code>	Token sin comillas y sin escribir Bearer.
<code>cancionId</code>	<code>1, 2, 3...</code>	ID real devuelto al crear una

Variable	Valor de ejemplo	Uso
		canción.

Regla para el token

Copia únicamente el valor del access_token, desde 1| hasta el último carácter. No copies las comillas y no escribas Bearer manualmente. Bruno agrega Bearer al seleccionar Auth > Bearer Token.

Encabezados que se usarán con frecuencia:

```
Accept: application/json
Content-Type: application/json
```

3. Pruebas de autenticación en Bruno

A1

Registrar usuario

POST {{baseUrl}}/register | Esperado: 201 Created

1. En Auth selecciona No Auth.
2. En Headers agrega Accept y Content-Type con valor application/json.
3. En Body selecciona JSON y pega el cuerpo mostrado abajo.
4. Usa un correo diferente en cada prueba de registro.

Body JSON

```
{
  "name": "Edsai Garcia",
  "email": "edsai.canciones13@example.com",
  "password": "Canciones2026#",
  "password_confirmation": "Canciones2026#"
}
```

Resultado esperado

La respuesta debe ser 201 Created e incluir user y authentication.access_token. Guarda el token en la variable token de Bruno.

A2

Iniciar sesión

POST {{baseUrl}}/login | Esperado: 200 OK

5. En Auth selecciona No Auth.
6. En Body > JSON utiliza el correo y la contraseña del usuario registrado.
7. Copia el nuevo access_token sin comillas y reemplaza la variable token si deseas usar este token.

Body JSON

```
{
  "email": "edsai.canciones13@example.com",
  "password": "Canciones2026#"
}
```

A3

Consultar usuario autenticado

GET {{baseUrl}}/me | Esperado: 200 OK

8. En Auth selecciona Bearer Token.
9. En Token escribe {{token}}.
10. No agregues Body.
11. Presiona Send y comprueba que aparezcan los datos del usuario.

Prueba negativa

Si quitas el token, colocas comillas o usas uno revocado, la respuesta correcta es 401 Unauthorized.

A4

Cerrar sesión

POST {{baseUrl}}/logout | Esperado: 200 OK

12. En Auth usa Bearer Token con {{token}}.
13. No agregues Body.
14. Presiona Send una sola vez. El token actual será eliminado.
15. Vuelve a ejecutar GET /me con el mismo token: ahora debe responder 401 Unauthorized.

Varios tokens

Registrar e iniciar sesión pueden generar tokens diferentes para el mismo usuario. Cerrar sesión elimina únicamente el token utilizado en esa solicitud.

4. Pruebas CRUD de canciones en Bruno

C1

Crear canción

POST {{baseUrl}}/canciones | Esperado: 201 Created

16. En Auth selecciona Bearer Token y escribe {{token}}.
17. En Headers agrega Accept y Content-Type con valor application/json.
18. En Body > JSON pega los datos completos de la canción.
19. No envíes user_id: Laravel lo obtiene automáticamente del token.

Body JSON

```
{
  "titulo": "Amanecer en la Sierra",
  "artista": "Edsai Garcia",
  "album": "Raíces de Oaxaca",
  "genero": "Regional Mexicano",
  "compositor": "Edsai Garcia",
  "sello_discografico": "Garrey Music",
  "fecha_lanzamiento": "2026-07-22",
  "duracion_segundos": 245,
  "numero_pista": 2,
  "idioma": "Español",
  "bpm": 102,
  "tonalidad": "Fa mayor",
  "precio": 22.50,
  "calificacion": 4.7,
  "reproducciones": 85000,
  "explicita": false,
  "disponible": true
}
```

Guardar el ID

Busca data.id en la respuesta y coloca ese número en la variable cancionId. Si la canción anterior fue eliminada,

el nuevo ID puede ser 2 o mayor.

C2

Listar canciones

GET `{{baseUrl}}/canciones` | Esperado: 200 OK

- 20. En Auth utiliza Bearer Token con `{{token}}`.
- 21. En Headers agrega Accept: application/json.
- 22. No agregues Body.
- 23. La respuesta contiene data, links y meta porque el listado está paginado.

C3

Consultar una canción

GET `{{baseUrl}}/canciones/{{cancionId}}` | Esperado: 200 OK

- 24. Verifica que `cancionId` tenga el ID real de una canción existente.
- 25. Usa Bearer Token con `{{token}}`.
- 26. La respuesta debe mostrar una sola canción con todos sus atributos.

C4

Actualizar parcialmente una canción

PATCH `{{baseUrl}}/canciones/{{cancionId}}` | Esperado: 200 OK

- 27. Usa Bearer Token con `{{token}}`.
- 28. Agrega Accept y Content-Type como application/json.
- 29. Envía únicamente los atributos que deseas modificar.

Body JSON

```
{
  "titulo": "Amanecer en la Sierra - Versión Especial",
  "precio": 24.90,
  "calificacion": 5,
  "reproducciones": 150000,
  "disponible": true
}
```

Comprobación

Después del PATCH ejecuta GET C3. `created_at` debe permanecer igual y `updated_at` debe cambiar.

C5

Validación de datos incorrectos

POST `{{baseUrl}}/canciones` | Esperado: 422 Unprocessable Entity

- 30. Mantén un token válido.
- 31. Envía intencionalmente datos vacíos o fuera de rango.
- 32. La API debe rechazar la petición y mostrar el objeto errors.

Body JSON incorrecto

```
{
  "titulo": "",
  "artista": "",
  "genero": "",
  "duracion_segundos": 0,
  "idioma": "",
  "bpm": 500,
  "precio": -20,
}
```

```
"calificacion": 8,  
"explicita": false,  
"disponible": true  
}
```

Interpretación del 422

No es una falla del servidor. Demuestra que Laravel valida los campos antes de guardar información incorrecta.

C6

Eliminar canción

DELETE {{baseUrl}}/canciones/{{cancionId}} | Esperado: 200 OK

- 33. Usa Bearer Token con {{token}}.
- 34. No agregues Body.
- 35. Comprueba que la URL termine con el ID; no debe terminar solamente en /canciones/.
- 36. Después ejecuta GET C3: debe responder 404 Not Found.

5. Códigos de respuesta que debes explicar

Código	Significado
200 OK	Consulta, actualización, login o logout realizados correctamente.
201 Created	Se creó un usuario o una canción.
401 Unauthorized	Falta el token, es incorrecto o fue revocado.
404 Not Found	El recurso o la ruta con ese ID no existe.
405 Method Not Allowed	Se usó un método incorrecto o falta el ID en la URL.
422 Unprocessable Entity	Los datos no cumplen las validaciones.
500 Internal Server Error	Existe una excepción interna que debe revisarse en storage/logs/laravel.log.

6. Verificación directa en la base de datos MySQL

Estas consultas permiten demostrar que las acciones realizadas desde Bruno se guardan realmente en MySQL dentro del VPS.

- 37. Abre una terminal SSH conectada al VPS.
- 38. Entra a MySQL usando el usuario de la API.
- 39. Escribe la contraseña cuando MySQL la solicite; los caracteres no se muestran en pantalla.

Entrar a la base de datos

```
mysql -h 127.0.0.1 -u eagr_canciones -p eagr_canciones_api
```

6.1 Comprobar la base seleccionada y las tablas

```
SELECT DATABASE();  
SHOW TABLES;
```

- SELECT DATABASE() debe mostrar eagr_canciones_api.
- SHOW TABLES debe incluir users, canciones, personal_access_tokens y migrations.

6.2 Ver usuarios registrados desde Bruno

```
SELECT id, name, email, created_at  
FROM users  
ORDER BY id DESC;
```

Después de POST /register debe aparecer el nuevo usuario. La contraseña no se consulta porque está almacenada con hash.

6.3 Ver canciones creadas o actualizadas

```
SELECT id, user_id, titulo, artista, genero, precio,  
       calificacion, reproducciones, disponible,  
       created_at, updated_at  
FROM canciones  
ORDER BY id DESC;
```

- Después de POST /canciones debe aparecer una fila nueva.
- Después de PATCH deben cambiar los atributos y updated_at.
- Después de DELETE la fila correspondiente debe desaparecer.

6.4 Consultar una canción específica

```
SELECT *  
FROM canciones  
WHERE id = 2;
```

Cambia 2 por el valor real almacenado en cancionId.

6.5 Ver tokens de Sanctum

```
SELECT id, tokenable_id, name, last_used_at, created_at  
FROM personal_access_tokens  
ORDER BY id DESC;
```

- tokenable_id indica a qué usuario pertenece el token.
- last_used_at muestra la última vez que el token fue utilizado.
- NULL significa que el token fue creado, pero todavía no se ha usado.
- MySQL no muestra el token legible; Sanctum almacena una versión protegida.

6.6 Comprobar cuántos tokens activos tiene el usuario

```
SELECT COUNT(*) AS tokens_activos  
FROM personal_access_tokens  
WHERE tokenable_id = 1;
```

Después de POST /logout el total debe disminuir en uno, porque se elimina únicamente el token utilizado en esa petición.

6.7 Salir de MySQL

```
EXIT;
```

No uses migrate:fresh para una demostración con datos

php artisan migrate:fresh --force elimina todas las tablas y los datos. Solo se utiliza al reconstruir una base nueva o cuando se acepta perder la información.

7. Secuencia recomendada para exponer ante la maestra

Paso	Prueba	Qué explicar
1	GET /api	Demostrar que el servidor está en línea.
2	POST /register	Registrar un usuario y mostrar el 201 Created.
3	GET /me	Probar que el token identifica al usuario.
4	POST /canciones	Crear una canción con todos sus atributos.
5	GET /canciones	Listar las canciones del usuario.
6	GET /canciones/{id}	Consultar una canción específica.
7	PATCH /canciones/{id}	Modificar título, precio o reproducciones.
8	POST inválido	Mostrar el 422 y explicar las validaciones.
9	Consultas MySQL	Comprobar users, canciones y tokens.
10	DELETE /canciones/{id}	Eliminar la canción y comprobar 404.
11	POST /logout	Revocar el token y comprobar 401 en /me.

Explicación breve para decir en voz alta

Guion sugerido

La aplicación cliente Bruno envía peticiones HTTP a la API alojada en el VPS. Laravel valida los datos, autentica al usuario mediante Sanctum y utiliza Eloquent para guardar o consultar información en MySQL. Cada respuesta incluye un código HTTP que permite comprobar si la operación fue correcta, rechazada por validación, no autorizada o no encontrada. Finalmente, verifico en MySQL que los usuarios, canciones y tokens

realmente se almacenaron en el servidor.

8. Solución rápida de errores comunes

Error	Corrección
401 Unauthenticated	Revisa que el token esté sin comillas, que el ambiente VPS esté seleccionado y que Auth use Bearer Token con {{token}}.
404 route ... could not be found	Verifica que la URL contenga /api y el ID correspondiente.
405 DELETE not supported for api/canciones	Falta {{cancionId}} al final de la URL.
422 campos requeridos	Revisa Body > JSON, las comillas, los dos puntos y que Content-Type sea application/json.
500 Server Error	Consulta tail -n 100 storage/logs/laravel.log en el VPS.
ETIMEDOUT	La red bloquea el puerto 8082 o el firewall aún no permite la conexión.
data vacío	La consulta funciona, pero todavía no existen canciones para ese usuario.

Checklist final

- ☐ Ambiente VPS seleccionado.
- ☐ baseUrl configurada con /api.
- ☐ token guardado sin comillas.
- ☐ cancionId coincide con un registro real.
- ☐ Cada petición tiene el método correcto.
- ☐ POST y PATCH usan Body JSON y Content-Type.
- ☐ GET y DELETE no necesitan Body.
- ☐ Se guardaron las peticiones con el icono de disquete.
- ☐ Se verificaron users, canciones y personal_access_tokens en MySQL.

Resultado final esperado

La colección queda lista para demostrar registro, login, autenticación, CRUD completo, validación, consulta directa en MySQL y cierre de sesión.